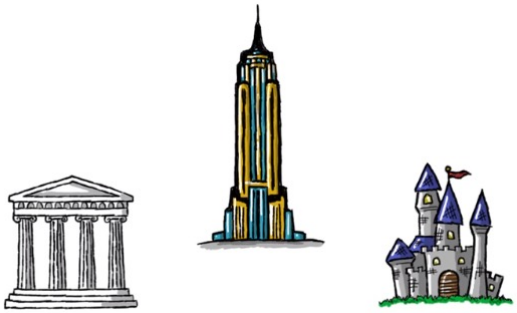
Durum Diyagramı Modelleme Yöntemi

1. Kullanım senaryolarını hazırla
2. Görünen (harici) özellikleri (percepts) belirle
3. Görünen özellikleri durumlarla (eşzamanlı makinelerle) modelle
4. Harici kontrolleri ve uyaranları belirle
5. İlave durumlar ve geçişler/olaylar ile modelle
6. Uyaran/tepki ikililerini listele
7. Gerekli (ör. Zamanlayıcılar) iç durumları belirle
8. koordinasyon mekanizmalarını oluştur
9. Eylemleri/aktiviteleri ekle
10. Senaryolar ile sonuç makinesini geçerle



Yazılım Mimarisi

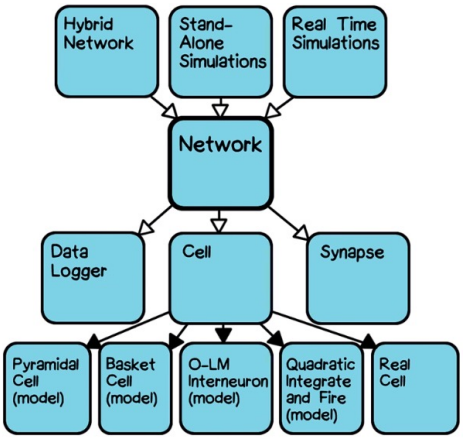
Yazılım Mimarisi – Genel Bakış

Bir tasarım probleminin en üst seviyeli ifadesi

Pratikte; kutular ve oklarla bir sistemin temel bileşenlerinin neler olduğunu ve birbirlerine nasıl bağlı olduklarını tasvir eden çizimlerdir.

- Kutular bileşenleri, oklar aralarındaki bağlantıları gösterir. Kontrol akışı ya da veri akışı olabilir.

Evrensel olarak kabul gören anlamsal bir uzlaşı sağlanmamıştır.



Gayresmi Tanım

Sistemin bileşen altsistemleri ya da modüller halinde organizasyonu ya da bölümlenmesi

Evrensel olarak mimari, katmanlar halinde gerçekleştirilir. İteratif olarak çoklu katmanlar iyileştirilir.

- En soyut halini oluşturan bileşenler
- Bu bileşenlerin alt bileşenleri / alt sistemleri
- Parçaların hakikaten gerçekleştirilebileceği kadar düşük bir seviyeye kadar devam ederiz

Genellikle klişe mimari stiller de kullanılır.

Analizin Bileşenlere Etkisi

Sadece analiz modeline bakarak sistemin bileşenlerini belirlemeye çalışalım.

Analiz modelleri;

Müşteri gereksinimlerindeki değişimlere iyi adapte olabilir.
- Tasarımdan önce yapıldıklarından, tasarım kısıtlarından etkilenmezler ve değişimlere adaptasyonları kolaydır.

Tasarımda kullanılacak yaklaşımı göstermemelidir.
- Analiz ve tasarımı mümkün olduğunca (etki altında bırakmamak için) bağımsız tutmaya çalışırız

Donanımdaki değişimlere karşı dirençli ve esnektir.
- Sistemin çalışacağı ortamla ilgili varsayımlar yapmamalıdır ve değişimlere ayak uydurabilmelidir.

Sistemin gerektirdiği tüm bileşenlere sahip değildir.
- Koleksiyon sınıfları ve benzeri işlem sınıfları tasarımda eklenecektir.

Resmi Tanım (USP'nin Tanımı)

The set of significant **decisions** about the organization of a software system,

the selection of the **structural elements** and their **interfaces** by which the system is composed,

together with their behavior as specified in the **collaborations** among those elements,

the **composition** of these structural and behavioral elements into progressively larger subsystems,

and the architectural **style** that guides this organization: these elements and their interfaces, their collaborations, and their composition.

Software architecture is concerned not only with structure and behavior but with **usage, functionality, performance, resilience, reuse, comprehensibility, economics and technology constraints and trade-offs, and aesthetic concerns**.

Resmi Tanım (USP'nin Tanımı)

Mimari, kararlarla ilgilidir.
- Mimarın seçimleri, hangi bileşenlerin olacağını, nasıl etkileşeceklerini, işlevsel olmayan gereksinimlerin nasıl karşılanacağını belirler.

Bileşenler söz konusu ise, bunlar yapısal elemanlar ve onların arayüzleridir (bulundukları dünyaya sundukları ve dünyadan bekledikleri)

Bileşenler birbirleriyle etkileşir.
- Diğer bileşenlerle işbirliği yaparlar

Mimari, yapısal ve davranışsal bu elemanların birleşip gitgide daha büyük alt sistemler ve sistemler oluşmasını sağlar.
- Bu birleşme mimari stillerin rehberliğinde (deneyim, daha önceden benzer sistemlerin geliştirilmesinde iyi reçeteler) gerçekleştirilebilir.

Yapısal ve davranışsal elemanlar dışında kullanım, performans, anlaşılabilirlik, ekonomi, teknolojik kısıtlar ve ödünler, estetik ile de ilgilidir.

Diğer Tanımlar

Dwayne Perry & Alex Wolf: Elements + forms + **rationale**
- Mantıksal temel: Mimarların yapmak için üzerinde anlaştıkları kararlar ve sebepler

IEEE: The fundamental **organization** of a system, embodied in its **components**, their **relationships** to each other and the environment, and the principles governing its design and evolution

Verhoff: The software architecture of a deployed software is determined by those aspects that are the hardest to **change**.
- Bileşenleri nelerin oluşturacağının tespiti, değişmesi en zor olan şeyleri saklamaya dayanır
- Bir sistem 5 yıl sonra aynı kalmayacak, değişecektir. Bu değişimler, sistemin beklenmedik şekillerde kırılmasına yol açacaktır.

Garlan & Shaw: Components + connectors + configurations

Mimariyi İfade Etmek

Component: Bileşen
- İşlemsel veya veri elemanı ve sistemin geri kalanı ile olan arayüzü (interface / port)
- Arayüz, sistemin kalanından bekledikleri ve sistemin kalanına sunduklarıdır

Connector: Bağlayıcı
- Bileşenler arasındaki haberleşme protokolleri
- Protokolü işletmek için kod da içerebilir

Configuration: Düzenleşim
- Bileşen ve bağlayıcıların nasıl bir araya geleceği
- İkili bir bağlayıcı ise, iki ucunda birer bileşen
- Tüm elemanların topolojisi konfigürasyonu oluşturur

Bileşenler

Taylor et al.: A software component is an architectural entity that
- (1) encapsulates a subset of the system's **functionality** and/or **data**
- (2) restricts access to that subset via an explicitly defined **interface**, and
- (3) has explicitly defined **dependencies** on its required execution context
  - Bileşenin istendiği şekilde çalışabilmesini sağlamak için neler gerekiyor?

Szyperski: A software component is a unit of **composition** (bileşenleri alıp bir araya getireceğiz) with **contractually** specified **interfaces** (arayüzlerimiz açık ve diğer bileşenler bunları bilir ve hemfikirdirler) and explicit context dependencies only. A software component can be deployed independently and is subject to composition by third parties.

Bileşenleri Seçmek

Bileşenleri seçmenin açık yolu, sistemin ne yapacağını / hesaplayacağını ortaya koymak ve bunu parçalara ayırmaktır. Burada bir belirsizlik de söz konusudur.

Bunların dışında pek çok faktör de hangi bileşenlerin sistemimizin parçası olacağını etkiler.

- Beklenen işlevsellik
- Halihazırda var olan (önceden yazdığınız/kütüphanelerden gelen) yeniden kullanılabilir parçalar
- Fiziksel bilgisayar mimarisi (ör. Çok çekirdekli işlemcide paralelleştirme)
- Ekibiniz ve deneyimi
- Conway's Law: Bir sistemin son hali, onu inşa eden organizasyonun yapısına bağlıdır.
- Sistemin değişim yörüngesi

Uygulama Programlama Arayüzü (API)

Bileşen tanımının bir kısmını oluşturur ve bir kısmını gerektirir

Bileşenin uygulama programlama arayüzü olarak tanımlanır.

- İndirdiğiniz bir uygulamadaki sınıflar, metotları, … bunların tanımı API'yi oluşturur
- Birimdeki elemanları nasıl çağırabileceğinize dair isimler (metot isimleri, argümanlar, argümanların tipleri, geri dönüş değeri, …)

API belirli bir programlama dili ile tanımlanabilir. (Dile Bağlama: Language binding)

- OCL gibi daha yüksek seviyeli soyutlama dili ile tanımlanabilir
- ADL (Architectural Description Language) gibi özelleştirilmiş notasyonlar da kullanılabilir

Bağlayıcılar (Connectors)

İşlevsellik (gereksinimlerden doğan) sorgusu ile bileşenleri bulmak daha kolaydır.

Bağlayıcılar biraz daha karışıktır.

- Tasarımcının belirli problemlerle başa çıkabilmek için bazı kararlar alması gerekir.

Taylor et al.: A software connector is an architectural element tasked with **effecting** and **regulating interactions** among components.

- Bileşenler arası akış (piping)

Bağlayıcılar, bileşenler arası etkileşim için bir protokol sağlar.

- Protokol: Kimin hangi sıra ile konuşacağını, hangi bilginin hangi yönde iletileceğini, yanlış bir şey olduğunda ne yapılacağını söyleyen bir dil.

Örnek Bağlayıcı

En basit bağlayıcı: Prosedür çağrısı / geri dönüşü

İki rol var : Çağıran ve çağrılan

Bu bir mesaj çiftidir. İlkinde bir metot çağrılır, ikincide geri döndürülen bir değer alınır.

Asimetrik ilişkidir. Çağıran çağrıyı başlatır ve çağrılanı bekler.

Senkron ilişkidir. Çağıran, çağrılan cevap verene kadar durur/işlem yapmaz.

Bağlayıcılar, tipi olan parametreler ve geri dönüş değeri ile bilginin aktarımına imkan tanır.

Düzenleşim (Configuration)

Bileşen ve bağlayıcılar olduğunda, bunları birleştirmek gerekir.

Buna konfigürasyon adı verilir.

Taylor et al.: An architectural configuration is a set of **specific associations between the components and connectors** of a software system's architecture.

Kalan Terminoloji

Kavramsal Mimari (Conceptual Architecture): Kavramsal demek, belirsiz ve yüksek seviyeli anlamına gelir. Geliştirme sürecinin en başlarında, -gereksinimlerin neler olduğu hakkında tam bir fikrimiz olmadan bile önce- üretilir.

- Çok soyut bir seviyede bileşenler ve bağlaıcıların neler olabileceğini düşünmemizle başlar. Takımların neler olabileceğini, son ürünü ne kadar sürede üretebileceğimizi öngörmekle devam eder.

Planlanan vs. inşa edilen mimari (hayaller / gerçekler)

- Planlama süreci boyunca takım mimarinin ne olacağını belirler ve belgeler.
- Uygulama gerçeklenirken, başka bir şey inşa edilebilir.
- İyileştirme sürecinde gerçekleştirme ekibi açık kaynaklı veya başka takımdan gelen ve geliştirme süresini çok kısaltan bir bileşene rastlayabilir. Ancak bu bileşen, planlanan sonuca uymuyor olabilir.
- Buna mimari sapma (architectural drift) adı verilir. Bakım sürecinde oluşursa, mimari erozyon olarak adlandırılır. Bakım ekibi hız ve anı kurtarmak adına planlanmayan değişiklikler yapabilir ve bunu genele (dokümantasyona) yansıtmaz.

Mimari Görünümler

Mimari tanım sadece bir diyagram değildir. Bir kararlar kümesidir.

Bu kümeyi tam olarak tanımlamak için pek çok diyagram ve belge üretilebilir.

Bunlara mimari görünümler (architectural views) adını veriyoruz.

Bu küme geniş ve pek çok yönlü derleme içerebilir. Pek çok mimari diyagram bir kısmını ifade eder.

Şu UML diyagramları katkı sağlayabilir:

- Sınıf, Bileşen, Barındırma, Nesne, Paket, Kullanım Senaryosu, Ardışıl, İletişim, Etkileşim

Mimari Stiller

Binalar gibi yazılım sistemleri de çeşit çeşittir. Bunlara stil adını veriyoruz,

Taylor et al.: Named collection of architectural design decisions that

- (1) are applicable in a given development context,
- (2) constain architectural design decisions that are specific to a particular system within that context,
- (3) and elicit beneficial qualities in each resulting system.

İsimlendirilmiş karar koleksiyonu

- Bu kararlar sistem spesifikasyonları ile tanımlı belirli koşullar altında uygundur
- Tasarım kararları olası bileşenlerin neler olduğunu ve etkileşimlerini belirli bir şekilde olmaya zorlar
- Bu stillerden, oluşturduğumuz sistemde yararlı katkılar elde etmemizi sağlar.

Mimari Stiller

Servis veren ve servis alan yazılım bileşenleri fiziksel olarak ayrıdır (ayrı makinelerde). Bu sayede iki taraf da bağımsız olarak ölçeklendirilebilir.

Servis veren bileşenler, alanlar kimliklerini belirtmediği sürece servis alanların kim olduğunu bilmez. Bu sayede şeffaf olarak değişebilecek istem yapanlara hizmet verebilirler.

Servis alanlar birbirinden izoledir, bu sayede bağımsız olarak ekleme, çıkarma, değişim yaşayabilirler. Servis alanlar sadece servis verenlere bağımlıdır.

Servise talep belirli bir değerin üzerinde çıktığında, servis verenler dinamik olarak eklenebilir ve mevcut hizmet verenlerin yükünü azaltabilir.

Hangisi?

Blackboard – Client Server – Implicit Invocation – Layered – Object Oriented – Pipe and Filter

Mimari Stiller

Faydalar:

Problem üzerindeki deneyimimizi kodlamış oluruz.

Örneğin Client server stilinde parçaları bölmek ve bağlamak için diğerlerinden daha iyi çalışacak pek çok seçeneğimiz bulunmaktadır. Ayrıca problemlerin neler olabileceğini ve bunlar çözmek için neler yapabileceğimizi de biliriz.

Bu sayede genel geliştirme yükümüz azalır. Çıkmaz sokaklara dalmayız.

Mimari stiller aynı zamanda standartlar haline de gelebilir. (Referans Mimari) Seçtiğimiz çözümün uygunluğunu geçerleyebilmemizi de sağlar.

Mimari stiller yeniden kullanımı da destekler. (Hazır bir mysql server kullanabiliriz)

Geliştirme sürecimizi de şekillendirebilir.

Stil Kataloğumuz

Yıllar içinde belirli durumlara çözüm olmak üzere oluşan uzun büyük bir listemiz var.

Tasarım deneyim gerektirir. Deneyim de, olası çözümlerden haberdar olmaktan geçer.

Abstract Data Type (ADT): Temsilin gizlenmesi

Batch sequential: Geçerle, düzenle, güncelle döngüsü

Blackboard: havuz, fırsatçı kontrol, işbirliği yapan ajanlar (istekler ve sonuçlar ortak veri havuzuna)

Big ball of mud: (aslında stil değil, stil eksikliği): Takım, mimari tasarım çalışması yapmadığında oluşur

Client-server: hareket işleme, çok katman

Stil Kataloğumuz

Component-based: iyi tanımlı arayüzlerle haberleşen tekrar kullanılabilir modüller

Coroutines: Simetrik etkileşim. A, B'yi, B, A'yı çağırabilir. A, B'yi 2.kere çağırdığında, B ilk çağrıda kaldığı yerden devam eder. (printf'teki format ve veri ikilisi gibi)

Data centric: Veritabanındaki saklı yordamların kullanılması

Domain Driven Design (DDD): iş odaklı, iş modeli. Veri akışı, iş modelinde otomatik değişikliklere neden olur. (Ör. Vergi hesabı uygulama programı)

Implicit Invocation: Olaylar, tepkiler, abone-yayım

Layered: soyut makineler, kısıtlı görünürlük. Her katman bir soyut makine gibi davranır. Üstündeki katmanlara hizmet eder (Ör. X11)

Daha Fazla Stil…

İlk stil: Master control: hiyerarşik olarak alttaki yordamları çağır ve geri dön

Message bus: asenkron mesaj ortak bir veriyolundan geçer

Mobile code: Özellikle mobil cihazlar için, istek üzerine kod, uzaktan değerlendirme, mobil ajan

Object oriented: Nesneye yönelik programlamadan biraz farklı. asenkron mesaj iletimi, bağımsız kontrol iş parçacığı. Yine nesnelerimiz var ama hepsinin varlığı bağımsız, kontrolleri kendilerinde. Birbirlerine asenkron mesaj yollayarak, işbirliği içinde problemi çözer.

Peer-to-peer: Eşit ortaklar servisin sorumluluğu paylaşır.

Plug-in: Kayıtlı üçüncü parti eklentiler (ör. Eclipse) Sisteme eklentiler yaparak büyütmek

Daha fazla Stil...

Pipe-and-filter: Tek yönlü, sıralı, girdi-çıktı, ASCII katarı

Process control: geri besleme döngüsü. Donanımı kontrol eden yazılım. (Ör. Araba hız sabitleme sistemi)

Production systems: (yapay zekadan) kural tabanı, koşullu çalışma. Sistemi gerçekleştirmek üzere kontrol akışı hakkında tam fikrimiz yoksa kullanabiliriz.

Representational state transfer (REST): ör: HTTP uygulamaları. istemci-sunucu ilişkisinde, katmanlı, durumsuz (stateless, her istek bağımsız ele alınır), önbelleğe alınabilir dağıtık hiperortamlar (performans için)

Daha fazla Stil...

Service-oriented (SOA): ;Sistemin işlevleri ayrık servislerde. Sistem bir servis kümesi. gevşek ilişkili, durumsuz, keşfedilebilir, kontrat-tanımlı

Shared nothing (Sharded): düğümler arası paylaşım olmayan dağıtık veritabanı.

State-transition systems: Reaktif, Asenkron olaylara tepki veren sistemler. Ör. GUI ve kullanıcının yarattığı olaylar. Gerçek-zamanlı da olabilir.

Shared memory: Senkronizasyon için kilit mekanizması. Tüm işlevler ortak bir bilgi havuzunu kullanır.

Table-driven interpreter: Çözümle ve Dağıt. İstekler belirli bir dil ile kodlanır ve bir yorumlayıcı ile bunlar çözümlenir ve ilgili yordama gönderilir.

Stil Sorunları

Sanki bütün problemleri stillerle çözmüşüz gibi konuşsak da, hala pek çok sorun mevcuttur.

Gerçek ve büyük sistemler birden fazla stile ihtiyaç duyar. Bunlara heterojen sistem adını veririz.

- İstemci/sunucu birimlerimiz, reaktif bir GUI, vb çeşitli yaklaşımların kombinasyonu

Bazı durumlar, mimari bir stile uysa da, çalışma alanına çok bağlıdır. Belirli bir askeri uçağı düşünün. Çeşitli versiyonları olacaktır. Ancak kontrol mekanziması çok benzer olacaktır. Farktan çok benzerlik içerir.

- Domain-specific software architecture (DSSA): Referans mimari de denir. Ortak işlevler burada tanımlanır, özelleşenler kendi mimarileri ile tanımlanır.

Anlam (Semantics). Mimari alanı ilerledikçe, stillerin tanımlarının daha kesin olmaları gerekir.

Architecture Description Language (ADL)

Mimariyi tanımlamak için notasyon

Sadece diyagrama sahip olmaktan az da olsa fazla resmiyet ve kesinlik sağlar

Araç desteği sağlar.
- Diyagram çizim aracı, stile uyup uymadığınızı kontrol eder.
- Çözümleyiciler (analyzers) sisteminizin yapısal ve davranışsal özelliklerini saptar.
- Simülatörler

Popüler Mimari Tanım dilleri:
- **Acme**, Wright, Rapide, ArTek, Derneter, CODE, Modechart, PSDL/CAPS, Resolve, UniCon

Mimari Değerlendirme

Acaba doğru mimariyi uygulayabildik mi?

Ürettiğimiz mimariyi doğruluk, bütünlük, tutarlılık ve diğer kalite unsurları açısından yargılayacağımız bir sürece ihtiyaç duyarız.

Eğer hata yaparsak, bir değişiklik yapmak maliyetli olacaktır.

Mimari gözden geçirme tahtası (Architectural review board): Büyük ve karmaşık sistemlerde, çoklu takımların oluşturduğu sistemlerde, hatta sistemlerin sistemlerinde bir yerde değişiklik yapmak, hiç beklenmedik bir kısımda beklenmedik bir etki yaratabilir. En iyisi, geliştiricilerin bir araya gelip bu tarz etkileri gözden geçirmesidir. Bunu periyodik olarak yapan kurumlar da vardır.

Software Architecture Assessment Method (SAAM): Biraz daha gayriresmi

Architecture Tradeoff Analysis Method (ATAM): Daha resmi. Carnegie Mellon Software Enigneering Institute'ta üretildi.

Software Architecture Assessment Method (SAAM)

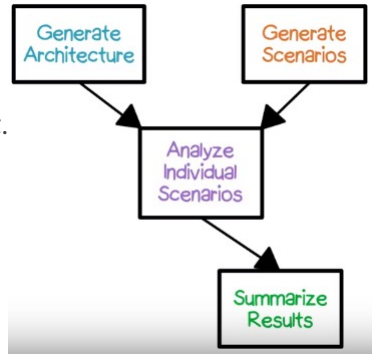
Mimari tasarım sürecinden geçip diyagram gibi çıktılar üretmiş olmalıyız. Mimariyi yarattık.

Senaryolar yaratırız. (Kullanım senaryosu değil, sistemin içinden bakarak detaylı senaryolar)

Kullanım senaryosu bir hesabın yapılması ise, burada mimarinin hangi elemanları beklenen işlevselliği sağlamak için hangi sırada gerekli onları ortaya koyarız.

Diyagramda adım adım yürüyerek, belirli kullanım mimariyi nasıl etkiler ortaya çıkarırırız. Özellikle yeni eklenen işlevlerin mimariyi nasıl etkilediği için yapılır.

- Tasarım alternatifleri de değerlendirilerek en az etkiyi yaratan seçilir.



Özet

Yazılım mimarisine ve Mimari stillere bir giriş yaptık.

Nihai hedefimiz, yüksek kaliteli sistemler üretmek ve maliyetleri azaltmak.

Bunu yapmanın en iyi yolu, problemlerin erkenden belirlenmesidir.

- Bunun için de sistem neye benzeyecek en baştan belirlemek gerekir. Sorunlar ve mantıklı çözümleri, ortaya koymak gerekir.

Elimizdeki hazır çözümleri/araçları çözümde kullanabilmeliyiz.

Tüm bunlarla, tüm paydaşlara hızlı ve etkin şekilde fikirlerimizi aktarabileceğimiz kadar soyut bir mimari inşa etmek isteriz.